Lab Assignment: Spam vs. Ham Text Classification with Feedforward Neural Networks

Natural Language Processing Course

2026

1 Course Overview & Objectives

In this assignment, you will build, train, and evaluate a Feedforward Neural Network (FNN) from scratch using PyTorch to solve a classic natural language processing task: **Spam vs. Ham (legitimate message) classification**.

By completing this assignment, you will learn how to:

1. Preprocess raw text data (tokenization, lowercasing, stop-word removal, and explicit data splitting).
2. Map discrete tokens into continuous vector spaces using PyTorch embedding layers.
3. Aggregate variable-length document text into fixed-size representations via feature averaging (mean pooling).
4. Construct, train, and evaluate a multi-layer neural network binary classifier using PyTorch.
5. Experiment with hyperparameters—specifically **learning rate** and **number of epochs**—to observe convergence, generalization, and how excessive training leads to memorizing training data ("learning by heart" or overfitting).

2 Dataset Description

We will use the **SMS Spam Collection** dataset, which contains SMS messages labeled as either **ham** (legitimate) or **spam** (junk/marketing). The dataset can be loaded programmatically from a public repository using Pandas:

- **URL:** `https://raw.githubusercontent.com/justmarkham/DAT8/master/data/sms.tsv`
- **Format:** Tab-separated values (`.tsv`) with two columns: `label` and `message`.

3 Core Architectural Concepts & Guidelines

3.1 1. The Word Embedding Average Approach (Mean Pooling)

Standard Feedforward Neural Networks require a fixed-size input vector for every sample, but text documents vary wildly in length. To bridge this gap:

- Each word index in a message is mapped by an embedding layer into a dense vector of size D_{embed}.
- For a message with N tokens, the embedding layer outputs a tensor of shape (batch_size, seq_len, D_{embed}).
- To collapse the variable sequence length, we apply **mean pooling (averaging)** across the sequence dimension:

$$\mathbf{x}_{\text{doc}} = \frac{1}{N} \sum_{i=1}^{N} \text{embedding}(w_i)$$

- In PyTorch, this is implemented as `embedded_tokens.mean(dim=1)`, reducing the tensor shape to (batch_size, D_{embed}).

3.2 2. Sizing Your Network Layers

- **Input Layer / First Linear Layer:** Because document representations are formed by averaging word embedding vectors, the input dimension to your first linear layer **must exactly equal your chosen embedding dimension** (`nn.Linear(embedding_dim, hidden_dim)`).
- **Hidden Layer(s):** A tunable feature transformer (common choices: 64 or 128 neurons) with a non-linear activation (e.g., `nn.ReLU()`).
- **Output Layer:** Because this is binary classification trained with `nn.BCEWithLogitsLoss()`, your final linear layer should have exactly **1 output neuron** (`output_dim = 1`) to output a raw score (logit).

3.3 3. Training Dynamics: Learning Rate & Epochs

- **Learning Rate (η):** Controls the gradient descent step size. Start with $\eta = 0.001$ using the Adam optimizer.
- **Number of Epochs & Overfitting:** Train across multiple lengths (**5, 10, 20, and 40 epochs**). Watch how pushing epochs too high causes training loss to drop near zero while test performance plateaus or degrades—illustrating **"learning by heart"** (overfitting to training noise).

4 Step-by-Step Implementation Instructions

1. **Data Loading & Explicit Splitting:** Load the dataset, map labels (`spam` $\rightarrow$ 1, `ham` $\rightarrow$ 0), clean the text (lowercase, stop-word removal, punctuation stripping), and **explicitly split your dataset into training (80%) and testing (20%) sets** before building your vocabulary.
2. **Vocabulary Construction:** Build your vocabulary dictionary (`word2idx`) **strictly using the training set** to prevent data leakage. Include an `<UNK>` token for unseen words.
3. **Model Architecture Design:** Create a PyTorch module containing an embedding layer, a hidden linear layer matching `embedding_dim`, a ReLU activation, and a single-neuron output layer.

4. **Loss & Evaluation Note:** Use `nn.BCEWithLogitsLoss()` during training. During evaluation, remember that model outputs are raw logits; convert them to binary predictions by checking if the logit is ≥ 0.0 (or if sigmoid(logit) ≥ 0.5).

5. **Experimental Evaluation:** Write a training loop that tests epoch configurations of 5, 10, 20, and 40 epochs, recording both training and test metrics.

5 Starter Code Template

```
import torch
import torch.nn as nn
import torch.optim as optim
import pandas as pd
import nltk
from nltk.corpus import stopwords
import string
import random
from sklearn.model_selection import train_test_split

nltk.download('stopwords')

# 1. Load, Clean, and Split Data
def load_and_split_data():
    url = "https://raw.githubusercontent.com/justmarkham/DAT8/master/data/
        sms.tsv"
    df = pd.read_csv(url, sep='\t', header=None, names=['label', 'message'
        ])
    df['label_num'] = df['label'].map({'spam': 1, 'ham': 0})

    stop_words = set(stopwords.words('english'))
    punctuations = set(string.punctuation)

    processed_data = []
    for msg, label in zip(df['message'], df['label_num']):
        tokens = [w.lower() for w in msg.split() if w.lower() not in
            stop_words and w not in punctuations]
        processed_data.append((tokens, label))

    # Explicit 80/20 Train-Test Split
    train_data, test_data = train_test_split(processed_data, test_size
        =0.2, random_state=42)
    return train_data, test_data

train_dataset, test_dataset = load_and_split_data()

# 2. Build Vocabulary (Using Training Data Only)
# TODO: Build word2idx dictionary, incorporating '<UNK>' token.

# 3. Define Feedforward Network Model
class SpamClassifierFNN(nn.Module):
    def __init__(self, vocab_size, embedding_dim, hidden_dim, output_dim
        =1):
```

```
        super(SpamClassifierFNN, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embedding_dim)

        # Input features to fc1 must equal embedding_dim due to mean
            pooling!
        self.fc1 = nn.Linear(embedding_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
        embedded = self.embedding(x)
        doc_vector = embedded.mean(dim=1)  # Mean pooling across sequence
            length
        out = self.fc1(doc_vector)
        out = self.relu(out)
        out = self.fc2(out)
        return out

# 4. Training & Epoch Experiment Loop
# TODO: Set up BCEWithLogitsLoss and Adam optimizer (lr=0.001).
# Test epoch settings [5, 10, 20, 40] and track training vs test accuracy.
# Note: For accuracy, evaluate predictions as (logits >= 0).
```

6 Deliverables & Grading Criteria

1. **Working Python Script/Notebook:** Clean code implementing explicit data splitting, vocabulary building, mean-pooling FNN architecture, and multi-epoch evaluation.
2. **Experimental Report (1–2 pages):** A summary table or plot contrasting training vs. test accuracy across epoch settings [5, 10, 20, 40], complete with analysis answering how over-training leads to "learning by heart."
3. **Performance Target:** A stable test accuracy of at least 70%.

7 Grading Rubric (Total: 5.0 Points)

Your submission will be evaluated based on the following criteria:

1. **Data Preprocessing & Vocabulary (1.0 Point)**
   - **Explicit Split:** Dataset is correctly split into an 80% training set and a 20% test set *prior* to vocabulary construction.
   - **No Data Leakage:** The vocabulary (`word2idx`) is built strictly from the training partition, incorporating an `<UNK>` token for unseen words.
   - **Text Cleaning:** Proper implementation of lowercasing, stop-word removal, and punctuation stripping.
2. **Model Architecture & Mean Pooling (1.0 Point)**

- **FNN Structure:** Properly subclasses `nn.Module` with an embedding layer, hidden linear layer, non-linear activation (ReLU), and a single-neuron output layer (`output_dim = 1`).
- **Dimension Alignment:** The input dimension of the first linear layer correctly matches the chosen embedding dimension to accommodate sequence mean pooling (`embedded.mean(dim=1)`).

3. **Training Setup & Logic (1.0 Point)**

- **Loss & Optimizer:** Utilizes `nn.BCEWithLogitsLoss()` paired with the Adam optimizer ($\eta = 0.001$).
- **Inference Logic:** Accurately converts raw output logits into binary predictions using the correct threshold (≥ 0.0 or sigmoid ≥ 0.5).

4. **Experimentation & Overfitting Analysis (1.0 Point)**

- **Epoch Sweep:** Successfully trains and logs performance across all required epoch configurations [5, 10, 20, and 40].
- **Written Report:** The 1–2 page report clearly compares training vs. test accuracy trends and provides insightful analysis regarding "learning by heart" (overfitting).

5. **Code Quality & Performance Target (1.0 Point)**

- **Execution & Accuracy:** Code runs end-to-end without errors and achieves a stable test accuracy of at least 70%.
- **Readability:** Clean script organization, appropriate comments, and clear variable naming.